IF2224 Teori Bahasa Formal dan Otomata
Milestone 1

13524121 - Billy Ontoseno Irawan
13524122 - Nathaniel Christian
13524139 - Azri Arzaq Pohan
13524142 - Rasyad Satyatma

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026

BAB 1
Dasar Teori

1.1 *Deterministic Finite Automata* (DFA)

Deterministic Finite Automata (DFA) adalah model matematis dalam teori komputasi yang digunakan untuk mengenali bahasa reguler. DFA didefinisikan secara formal sebagai

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q merupakan himpunan state (keadaan) yang terbatas, mewakili kondisi mesin pada saat tertentu
- Σ adalah alfabet input, yaitu himpunan simbol yang dapat diproses
- $\delta : Q \times \Sigma \rightarrow Q$ adalah fungsi transisi yang menentukan state selanjutnya secara unik berdasarkan state saat ini dan simbol input
- $q_0 \in Q$ adalah state awal, titik mulai pemrosesan
- $F \subseteq Q$ adalah himpunan state akhir yang menandakan penerimaan input jika proses berakhir di salah satunya.

Sifat deterministik DFA memastikan bahwa untuk setiap pasangan *state* dan input, hanya ada satu transisi sehingga menghindari ambiguitas dan memungkinkan simulasi efisien dengan kompleksitas waktu linier $O(n)$ dengan n sebagai panjang string input. Aplikasi DFA meliputi pengenalan pola dalam string seperti dalam mesin pencari atau validator input, karena kemampuannya memproses data secara sekuensial tanpa *backtracking*.

1.2 *Lexical Analyzer* (Lexer)

Lexer atau *lexical analyzer* adalah komponen awal dalam proses kompilasi yang bertugas mengubah kode sumber dari format karakter mentah menjadi unit-unit sintaksis kecil yang dikenal sebagai token. Lexer dapat diimplementasikan menggunakan automata seperti DFA untuk mengenali pola reguler; fungsi transisi DFA diimplementasikan sebagai pemrosesan karakter demi karakter. Lexer membaca kode sumber per karakter dari kiri ke kanan dan mengelompokkan karakter-karakter tersebut menjadi token berdasarkan pola tertentu yang didefinisikan oleh bahasa pemrograman. Setiap token memiliki dua komponen utama yaitu tipe (kategori atau kelas dari token tersebut) serta nilai (sekuens karakter yang membentuk token tersebut). Dengan menyederhanakan kode melalui tokenisasi, lexer memudahkan deteksi pola dan struktur bahasa pada tahap parsing. Selain itu, lexer menjaga ketelitian dengan memastikan bahwa input kode sumber sesuai dengan aturan bahasa sebelum analisis lebih lanjut dimula

BAB 2
Perancangan dan Implementasi

Proses *lexical analysis* (lexer) berfungsi untuk melakukan analisis leksikal dan untuk bahasa pemrograman Arion ini, proses tersebut kami implementasikan menggunakan bahasa C++ dengan mengikuti paradigma *Object-Oriented Programming*. Fokus utama yang menjadi bahasan pada bab ini mengenai perancangan DFA antar state, serta implementasi dari *lexer*.

2.1 Bahasa Pemrograman

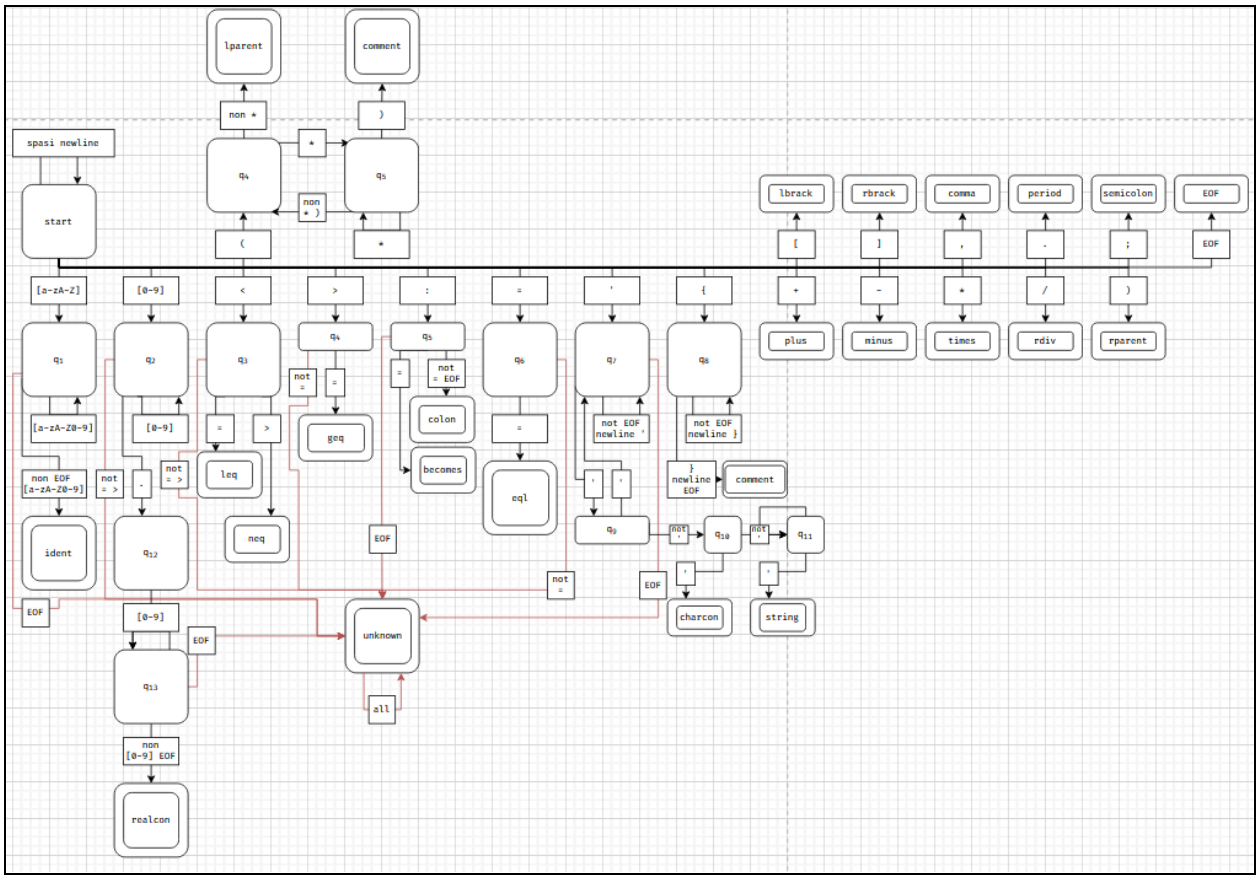
Implementasi dari lexer yang digunakan pada program ini menggunakan bahasa pemrograman C++ dengan version C++11 dan dikompilasi menggunakan g++ melalui Makefile. Kami memilih bahasa ini karena C++ menggunakan paradigma *Object-Oriented Programming* secara native melalui class, struct, dan enum class. Program ini juga menggunakan komponen STL seperti std::unordered_map untuk melakukan pencarian berdasarkan keyword dengan kompleksitas O(1) dan std::string untuk melakukan manipulasi string yang dapat membantu dalam implementasi dari tabel keyword dan juga logika *scanning*.

Struktur pada program ini dirancang secara modular untuk pembagian tanggung jawab yang jelas, seperti enum TokenType mendefinisikan seluruh jenis token yang dapat dikenali oleh program. Struct Token merepresentasikan entitas dari token. Kelas Lexer mengintegrasikan kedua komponen tersebut sekaligus meng-*handle* logika *scanning* dan juga transisi dari DFA.

2.2 Diagram DFA

Seluruh jenis token yang terdapat dalam spesifikasi Bahasa Arion dirancang dalam sebuah Diagram DFA. Setiap jenis token memiliki pola transisi *state* yang unik, dimulai dari *state* awal dan berakhir pada *state* akhir yang merepresentasikan jenis token tertentu. Process dari *scanning* dilakukan per karakter, dimana setiap karakter input akan memicu sebuah transisi dari satu *state* ke *state* lainnya berdasarkan aturan yang telah didefinisikan.

Untuk menentukan jenis token dari sekumpulan karakter yang telah diinput, lexer menggunakan satu DFA terintegrasi yang di *hardcode* ke dalam metode-metode kelas Lexer. Proses dari *scanning* dimulai dari *state* awal di dalam Metode getNextToken(), kemudian melakukan transisi *state* demi *state* yang mengikuti karakter input melalui pemanggilan metode-metode yang telah dibuat secara khusus untuk menanganinya seperti identOrKeyword(), number(), stringOrChar(), dan lain-lain. Jika bertemu karakter yang tidak valid, maka karakter tersebut akan dikategorikan sebagai token yang tidak dikenali (UNKNOWN).



Gambar 1. Diagram DFA

2.3 Enum TokenType dan Struct Token

Berikut implementasi dari Enum TokenType dan Struct Token:

(inside) ./src/Lexer/Lexer.hpp

```
enum class TokenType {
    INTCON, REALCON, CHARCON, STRING,   // Literal
    NOTSY, PLUS, MINUS, TIMES, IDIV, RDIV,   // Operator
    IMOD, ANDSY, ORSY,
    EQL, NEQ, GTR, GEQ, LSS, LEQ,   // Relational
    LPARENT, RPARENT, LBRACK, RBRACK,      // Punctuation
    COMMA, SEMICOLON, PERIOD, COLON, BECOMES,
    CONSTSY, TYPESY, VARSY, FUNCTIONSY,    // Keyword
    PROCEDURESY, ARRAYSY, RECORDSY, PROGRAMSY,
    IDENT, BEGINSY, IFSY, CASESY, REPEATSY,
    WHILESY, FORSY, ENDSY, ELSESY, UNTILSY,
    OFSY, DOSY, TOSY, DOWNTOSY, THENSY,
    COMMENT, END_OF_FILE, UNKNOWN         // Lain-lain
};

struct Token {
    TokenType type;
    std::string value;
```

```
    int line;
    int column;
};
```

Implementasi di atas menunjukkan struktur data yang digunakan untuk merepresentasikan token dari hasil *lexical analysis*. TokenType didefinisikan sebagai enum class yang mencakup seluruh 52 jenis token yang sesuai dengan spesifikasi dari bahasa Arion. Struct Token menyimpan 4 atribut penting: type yang digunakan untuk meng-identify jenis token seperti INTCON, IDENT, ataupun BEGINSY; value yang berisi *lexeme* atau string literal dari token tersebut; serta line dan column yang mencatat token dalam source code yang berguna untuk *error reporting*.

Ketika terjadi kesalahan sintaks atau semantik pada tahap kompilasi selanjutnya, *compiler* dapat menampilkan pesan *error* yang secara spesifik menunjukkan lokasi persis dari token yang bermasalah dalam kode program karena program telah menyimpan informasi dari posisi.

2.4 Kelas Lexer

Berikut implementasi dari kelas Lexer:

(inside) ./src/Lexer/Lexer.hpp

```
class Lexer {
    private:
        std::string source;
        int currPos;
        char currChar;
        int currLine;
        int currCol;
        std::unordered_map<std::string, TokenType> keywords;

        void advance();
        char peek();
        void skipWhiteSpace();
        std::string toLowerCase(const std::string& str);

        Token number();
        Token identOrKeyword();
        Token stringOrChar();
        Token commentCurly();
        Token commentParentheses();
    public:
        Lexer(const std::string& filename);
        Token getNextToken();
        std::string tokenTypeToString(TokenType type);
};
```

./src/Lexer/Lexer.cpp

```
#include "Lexer.hpp"
#include <iostream>
#include <sstream>
#include <cctype>
#include <fstream>
#include <stdexcept>
using namespace std;


Lexer::Lexer(const string& filename) : currPos(0), currLine(1), currCol(1) {
    ifstream file(filename);
    if (!file.is_open()) {
        throw runtime_error(filename + " tidak bisa dibuka.");
        currChar = '\0';
        return;
    }
    stringstream buffer;
    buffer << file.rdbuf();
    source = buffer.str();


    if (source.length() > 0) {
        currChar = source[0];
    } else {
        currChar = '\0';
    }


    keywords["const"] = TokenType::CONSTSY;
    keywords["type"] = TokenType::TYPESY;
    keywords["var"] = TokenType::VARSY;
    keywords["function"] = TokenType::FUNCTIONSY;
    keywords["procedure"] = TokenType::PROCEDURESY;
    keywords["array"] = TokenType::ARRAYSY;
    keywords["record"] = TokenType::RECORDSY;
    keywords["program"] = TokenType::PROGRAMSY;
    keywords["begin"] = TokenType::BEGINSY;
    keywords["if"] = TokenType::IFSY;
    keywords["case"] = TokenType::CASESY;
    keywords["repeat"] = TokenType::REPEATSY;
    keywords["while"] = TokenType::WHILESY;
    keywords["for"] = TokenType::FORSY;
    keywords["end"] = TokenType::ENDSY;
    keywords["else"] = TokenType::ELSESY;
    keywords["until"] = TokenType::UNTILSY;
    keywords["of"] = TokenType::OFSY;
    keywords["do"] = TokenType::DOSY;
    keywords["to"] = TokenType::TOSY;
    keywords["downto"] = TokenType::DOWNTOSY;
    keywords["then"] = TokenType::THENSY;
    keywords["not"] = TokenType::NOTSY;
    keywords["and"] = TokenType::ANDSY;
    keywords["or"] = TokenType::ORSY;
    keywords["mod"] = TokenType::IMOD;
```

```
    keywords["div"] = TokenType::IDIV;
}


void Lexer::advance() {
    currPos++;
    if (currPos >= (int)source.length()) {
        currChar = '\0';
    }
    else {
        currChar = source[currPos];
        currCol++;
        if (currChar == '\n') {
            currLine++;
            currCol = 0;
        }
    }
}


char Lexer::peek() {
    if (currPos + 1 >= (int)source.length()) {
        return '\0';
    }
    else {
        return source[currPos + 1];
    }
}


void Lexer::skipWhiteSpace() {
    while (currChar != '\0' && isspace(currChar)) {
        advance();
    }
}


string Lexer::toLowerCase(const string& str) {
    string lowerStr = str;
    for (char& c : lowerStr) {
        c = tolower(c);
    }
    return lowerStr;
}


Token Lexer::identOrKeyword() {
    string result = "";
    int startCol = currCol;
    while (currChar != '\0' && isalnum(currChar)) {
        result += currChar;
        advance();
    }
    string lowerResult = toLowerCase(result);
    TokenType type = TokenType::IDENT;
```

```
    if (keywords.find(lowerResult) != keywords.end()) {
        type = keywords[lowerResult];
    }
    return {type, result, currLine, startCol};
}


Token Lexer::number() {
    string result = "";
    int startCol = currCol;
    while (currChar != '\0' && isdigit(currChar)) {
        result += currChar;
        advance();
    }


    if (currChar == '.') {
        if (isdigit(peek())) {
            result += currChar;
            advance();
            while(currChar != '\0' && isdigit(currChar)) {
                result += currChar;
                advance();
            }
            return {TokenType::REALCON,result, currLine, startCol};
        }
    }
    return {TokenType::INTCON, result, currLine, startCol};
}


Token Lexer::stringOrChar() {
    string result = "";
    int startCol = currCol;


    advance();
    while (currChar != '\0' && currChar != '\n') {
        if(currChar == '\'') {
            if(peek() == '\'') {
                result += '\'';
                advance();
                advance();
            }
            else {
                advance();
                break;
            }
        }
        else {
            result += currChar;
            advance();
        }
    }
```

```
    TokenType type;
    if(result.length() == 1) {
        type = TokenType::CHARCON;
    }
    else {
        type = TokenType::STRING;
    }


    return {type, result, currLine, startCol};
}


Token Lexer::commentCurly() {
    string result = "{";
    int startCol = currCol;

    advance();
    while(currChar != '\0' && currChar != '}' && currChar != '\n') {
        result += currChar;
        advance();
    }


    if(currChar == '}') {
        result += '}';
        advance();
    }
    return {TokenType::COMMENT, result, currLine, startCol};
}


Token Lexer::commentParentheses() {
    string result = "(*";
    int startCol = currCol;
    advance();
    advance();
    while(currChar != '\0') {
        if(currChar == '*' && peek() == ')') {
            result += "*)";
            advance();
            advance();
            break;
        }
        result += currChar;
        advance();
    }
    return{TokenType::COMMENT, result, currLine, startCol};
}


Token Lexer::getNextToken() {
    while(currChar != '\0') {
        if(isspace(currChar)) {
            skipWhiteSpace();
```

```
            continue;
        }
        int startCol = currCol;
        if(isalpha(currChar)) {
            return identOrKeyword();
        }
        if(isdigit(currChar)) {
            return number();
        }
        if(currChar == '<') {
            advance();
            if(currChar == '=') {
                advance();
                return {TokenType::LEQ, "<=", currLine, startCol};
            }
            if(currChar == '>') {
                advance();
                return {TokenType::NEQ, "<>", currLine, startCol};
            }
            return {TokenType::LSS, "<", currLine, startCol};
        }
        if(currChar == '>') {
            advance();
            if(currChar == '=') {
                advance();
                return {TokenType::GEQ, ">=", currLine, startCol};
            }
            return {TokenType::GTR, ">", currLine, startCol};
        }
        if(currChar == ':') {
            advance();
            if(currChar == '=') {
                advance();
                return {TokenType::BECOMES, ":=", currLine, startCol};
            }
            return {TokenType::COLON, ":", currLine, startCol};
        }
        if(currChar == '=') {
            advance();
            if(currChar == '=') {
                advance();
                return {TokenType::EQL, "==", currLine, startCol};
            }
            return {TokenType::UNKNOWN, "=", currLine, startCol};
        }
        if(currChar == '\'') {
            return stringOrChar();
        }
        if(currChar == '{') {
            return commentCurly();
        }
        if(currChar == '(') {
            if(peek() == '*') {
                return commentParentheses();
            }
            advance();
```

```
            return {TokenType::LPARENT, "(", currLine, startCol};


        }


        char c = currChar;
        advance();
        if(c == '+') {
            return {TokenType::PLUS, "+", currLine, startCol};
        }
        else if(c == '-') {
            return {TokenType::MINUS, "-", currLine, startCol};
        }
        else if(c == '*') {
            return {TokenType::TIMES, "*", currLine, startCol};
        }
        else if(c == '/') {
            return {TokenType::RDIV, "/", currLine, startCol};
        }
        else if(c == ')') {
            return {TokenType::RPARENT, ")", currLine, startCol};
        }
        else if(c == '[') {
            return {TokenType::LBRACK, "[", currLine, startCol};
        }
        else if(c == ']') {
            return {TokenType::RBRACK, "]", currLine, startCol};
        }
        else if(c == ',') {
            return {TokenType::COMMA, ",", currLine, startCol};
        }
        else if(c == '.') {
            return {TokenType::PERIOD, ".", currLine, startCol};
        }
        else if(c == ';') {
            return {TokenType::SEMICOLON, ";", currLine, startCol};
        }
        else {
            string unknown(1, c);
            return {TokenType::UNKNOWN, unknown, currLine, startCol};
        }
    }
    return {TokenType::END_OF_FILE, "EOF", currLine, currCol};
}


string Lexer::tokenTypeToString(TokenType type) {
    if(type == TokenType::INTCON) {
        return "intcon";
    }
    else if(type == TokenType::REALCON) {
        return "realcon";
    }
    else if(type == TokenType::CHARCON) {
        return "charcon";
```

```
    }
    else if(type == TokenType::STRING) {
        return "string";
    }
    else if(type == TokenType::NOTSY) {
        return "notsy";
    }
    else if(type == TokenType::PLUS) {
        return "plus";
    }
    else if(type == TokenType::MINUS) {
        return "minus";
    }
    else if(type == TokenType::TIMES) {
        return "times";
    }
    else if(type == TokenType::IDIV) {
        return "idiv";
    }
    else if(type == TokenType::RDIV) {
        return "rdiv";
    }
    else if(type == TokenType::IMOD) {
        return "imod";
    }
    else if(type == TokenType::ANDSY) {
        return "andsy";
    }
    else if(type == TokenType::ORSY) {
        return "orsy";
    }
    else if(type == TokenType::EQL) {
        return "eql";
    }
    else if(type == TokenType::NEQ) {
        return "neq";
    }
    else if(type == TokenType::GTR) {
        return "gtr";
    }
    else if(type == TokenType::GEQ) {
        return "geq";
    }
    else if(type == TokenType::LSS) {
        return "lss";
    }
    else if(type == TokenType::LEQ) {
        return "leq";
    }
    else if(type == TokenType::LPARENT) {
        return "lparent";
    }
    else if(type == TokenType::RPARENT) {
        return "rparent";
    }
    else if(type == TokenType::LBRACK) {
```

```
        return "lbrack";
    }
    else if(type == TokenType::RBRACK) {
        return "rbrack";
    }
    else if(type == TokenType::COMMA) {
        return "comma";
    }
    else if(type == TokenType::SEMICOLON) {
        return "semicolon";
    }
    else if(type == TokenType::PERIOD) {
        return "period";
    }
    else if(type == TokenType::COLON) {
        return "colon";
    }
    else if(type == TokenType::BECOMES) {
        return "becomes";
    }
    else if(type == TokenType::CONSTSY) {
        return "constsy";
    }
    else if(type == TokenType::TYPESY) {
        return "typesy";
    }
    else if(type == TokenType::VARSY) {
        return "varsy";
    }
    else if(type == TokenType::FUNCTIONSY) {
        return "functionsy";
    }
    else if(type == TokenType::PROCEDURESY) {
        return "proceduresy";
    }
    else if(type == TokenType::ARRAYSY) {
        return "arraysy";
    }
    else if(type == TokenType::RECORDSY) {
        return "recordsy";
    }
    else if(type == TokenType::PROGRAMSY) {
        return "programsy";
    }
    else if(type == TokenType::IDENT) {
        return "ident";
    }
    else if(type == TokenType::BEGINSY) {
        return "beginsy";
    }
    else if(type == TokenType::IFSY) {
        return "ifsy";
    }
    else if(type == TokenType::CASESY) {
        return "casesy";
    }
```

```
    else if(type == TokenType::REPEATSY) {
        return "repeatsy";
    }
    else if(type == TokenType::WHILESY) {
        return "whilesy";
    }
    else if(type == TokenType::FORSY) {
        return "forsy";
    }
    else if(type == TokenType::ENDSY) {
        return "endsy";
    }
    else if(type == TokenType::ELSESY) {
        return "elsesy";
    }
    else if(type == TokenType::UNTILSY) {
        return "untilsy";
    }
    else if(type == TokenType::OFSY) {
        return "ofsy";
    }
    else if(type == TokenType::DOSY) {
        return "dosy";
    }
    else if(type == TokenType::TOSY) {
        return "tosy";
    }
    else if(type == TokenType::DOWNTOSY) {
        return "downtosy";
    }
    else if(type == TokenType::THENSY) {
        return "thensy";
    }
    else if(type == TokenType::COMMENT) {
        return "comment";
    }
    else if(type == TokenType::END_OF_FILE) {
        return "end_of_file";
    }
    else if(type == TokenType::UNKNOWN) {
        return "unknown";
    }
    else {
        return "error";
    }
}
```

Implementasi kode di atas menunjukkan bagaimana kelas Lexer diinisialisasi. Konstruktor dapat menerima nama dari file sebagai parameter lalu membaca seluruh isi dari file ke dalam std::string source, dan menginisialisasi posisi baca serta informasi dari baris dan kolom. Konstruktor juga membangun tabel *lookup* keyword dengan menggunakan std::unordered_map yang memetakan setiap keyword dari bahasa Arion (dalam *lowercase*) ke TokenType yang bersesuaian.

Metode advance() berfungsi untuk memajukan posisi baca satu karakter ke depan, sekaligus untuk memperbarui secara otomatis informasi dari currLine dan currCol. Karakter \0 digunakan sebagai *sentinel* untuk menandai akhir dari input.

Metode peek() menangani operator yang multi-karakter. Lexer perlu untuk mengintip karakter yang akan dikonsumsi, karena tanpa *lookahead*, lexer akan tidak dapat membedakan operator yang akan dikonsumsi, misalnya, `:` (COLON) dari `:=` (BECOMES).

Metode skipWhiteSpace() melewati seluruh karakter *whitespace* seperti spasi, tab, dan *newline* yang tidak memiliki makna leksikal.

Metode toLowerCase() digunakan untuk menormalisasi *case* agar pengecekan dari keyword bersifat *case-insensitive* sesuai dengan spesifikasi dari bahasa Arion. Sebagai contoh, BEGIN, Begin, dan begin semuanya harus dikenali sebagai keyword yang sama (BEGINSY).

Setiap kategori dari token juga memiliki metode *scanning* tersendiri yang merepresentasikan pola transisi *state* yang secara spesifik di-*handle*.

Metode identOrKeyword() mengenali *identifier* dan *keyword* dari DFA dimulai dari *state* awal ketika karakter pertama adalah huruf (isalpha). Selama karakter berikutnya berupa huruf atau digit (isalnum), DFA akan tetap berada dalam *state* pengumpulan karakter. Ketika bertemu dengan karakter yang bukan huruf ataupun digit, DFA akan bertransisi ke *accepting state*. Selanjutnya, setelah *lexeme* terbentuk maka akan dilakukan pencarian pada tabel keyword dengan menggunakan keywords.find(). Apabila ditemukan kecocokan, maka token tersebut akan dikategorikan sebagai keyword (misalnya BEGINSY, WHILESY) dan jika tidak, maka akan dikategorikan sebagai IDENT. Metode ini juga mengenali keyword logika dan aritmatika seperti not, and, or, mod, dan div. Meskipun secara semantik merupakan operator, tetapi secara leksikal keyword-keyword tersebut memiliki bentuk yang identik dengan identifier biasa, sehingga perlu melewati proses pengecekan tabel keyword untuk diklasifikasikan ke dalam TokenType yang tepat (seperti NOTSY, ANDSY, IDIV).

Metode number() mengenali konstanta bilangan, seperti INTCON maupun REALCON. DFA dari metode ini akan dimulai dari pengumpulan rangkaian digit. Metode ini akan menggunakan peek() untuk memeriksa karakter setelah titik adalah digit. Jika karakter setelah titik adalah digit, maka titik tersebut akan dibiarkan untuk diproses sebagai token terpisah, misalnya PERIOD pada akhiran end. dan bilangan akan dikembalikan sebagai INTCON. Dengan menggunakan *lookahead*, program akan dapat membedakan kasus seperti 3.14 (satu token REALCON) dengan end. (ENDSY dan PERIOD).

Metode stringOrChar() mengenali *string literal* dan *character literal*. DFA dari metode ini akan dimulai ketika karakter pembuka berupa tanda kutip tunggal `'`. Metode ini juga menangani *escaped quotes* selama berada di dalam *state* pengumpulan karakter. Ketika bertemu dengan dua tanda kutip berurutan, satu karakter kutip akan ditambahkan ke hasil dan DFAnya akan tetap berada di *state* pengumpulan. Lalu ketika bertemu tanda satu kutip tunggal yang bukan *escaped*, maka DFA akan bertransisi ke *accepting state*. Setelah semua karakter telah terkumpul, maka klasifikasi token akan ditentukan berdasarkan panjang dari *lexeme*: jika tepat

satu karakter, maka token akan dikategorikan sebagai CHARCON dan jika lebih dari satu akan dikategorikan sebagai STRING.

Metode commentCurly() dan commentParantheses() akan menangani dua bentuk komentar. Metode commentCurly() akan mengenali komentar yang diapit oleh kurung kurawal {...}, sedangkan commentParantheses() akan mengenali komentar yang diapit (*...*). Kedua metode ini menggunakan DFA yang mengumpulkan seluruh karakter di antara pembuka dan penutup komentar yang kemudian akan menghasilkan token COMMENT.

Metode commentParantheses() menggunakan teknik *lookahead* melalui peek() untuk mendeteksi pasangan penutup *) dan memastikan tanda bintang * yang bukan bagian dari penutup komentar.

Metode getNextToken() menentukan cabang transisi DFA mana yang akan dieksekusi oleh program berdasarkan karakter pertama yang dibaca. Proses tersebut akan dimulai dengan melewati *whitespace* dengan menggunakan skipWhiteSpace(). Untuk karakter huruf, DFA akan diarahkan ke identOrKeyword() dan untuk digit akan diarahkan ke number(). Untuk operator multi-karakter seperti <, >, :, dan =, metode ini akan melakukan transisi DFA secara inline dengan membaca karakter pertama lalu memeriksa karakter berikutnya untuk menentukan apakah operator tersebut merupakan satu karakter (misalnya < menjadi LSS) atau dua karakter (misalnya <= menjadi LEQ, <> menjadi NEQ). Untuk tanda kutip tunggal, akan diarahkan ke stringOrChar(). Untuk pembuka komentar { atau (*, DFA akan diarahkan ke metode komentar yang sesuai. Operator dan *punctuation* satu karakter akan langsung dikenali dan dikembalikan sebagai token. Jika karakter tidak cocok dengan pola manapun, maka DFA akan bertransisi ke *dead state* dan karakter akan dikategorikan sebagai token UNKNOWN, agar memastikan program tidak crash saat menemui input yang tidak valid. Ketika seluruh input telah diproses (karakter '\0'), maka selanjutnya metode ini akan mengembalikan token END_OF_FILE.

2.5 Alur Program

Berikut adalah kode main.cpp dari program:

./src/main.cpp

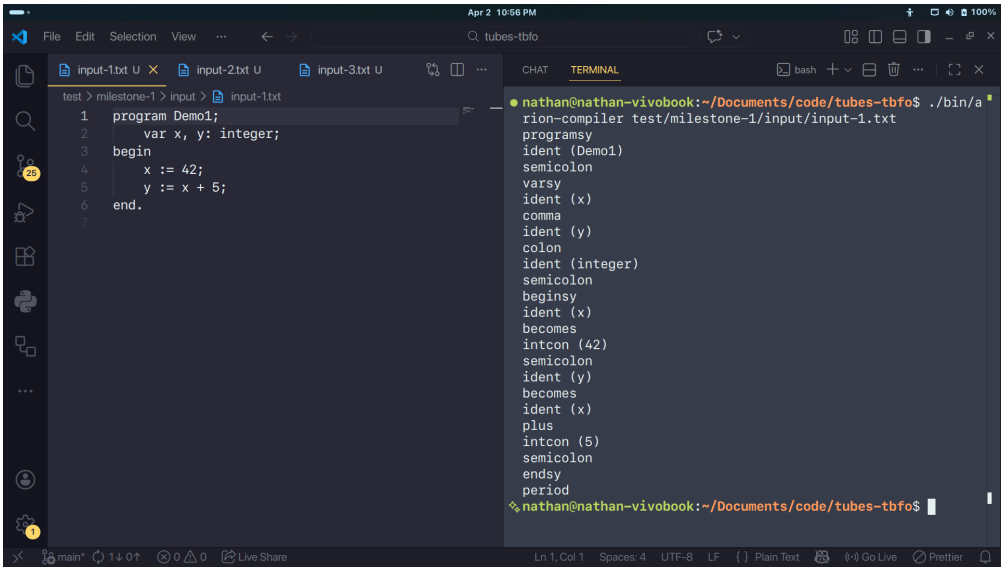
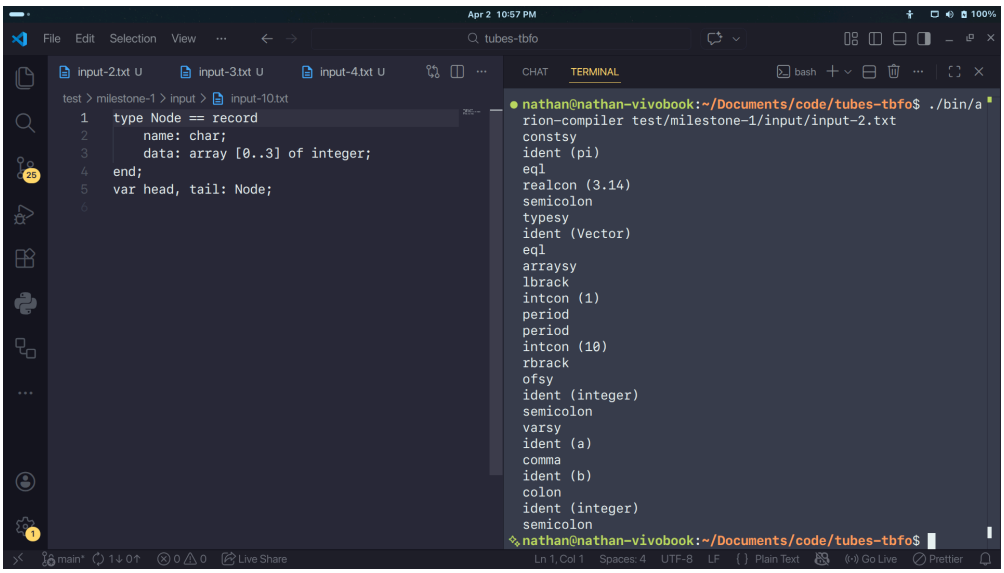
```
#include "Lexer/Lexer.hpp"
#include <string>
#include <iostream>
using namespace std;


int main(int argc, char* argv[]) {
    if(argc < 2) {
        cerr << "Format: '/bin/arion-compiler
/test/milestone-X/<input-file.txt>' atau bisa menggunakan 'make run'" << endl;
        return 1;
    }
    string filename = argv[1];
    try {
        Lexer lexer(filename);
        Token token = lexer.getNextToken();
        while(token.type != TokenType::END_OF_FILE) {
            string typeName = lexer.tokenTypeToString(token.type);

            if(token.type == TokenType::IDENT || token.type ==
```

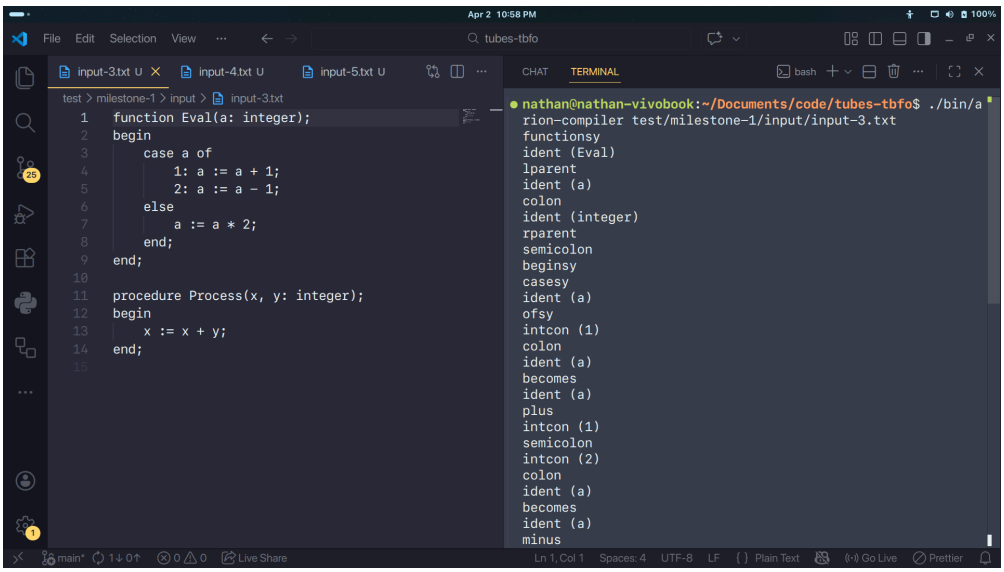
```
TokenType::INTCON || token.type == TokenType::REALCON) {
                cout << typeName << " (" << token.value << ")" << endl;
            }
            else if (token.type == TokenType::STRING || token.type ==
TokenType::CHARCON) {
                cout << typeName << " ('" << token.value << "')" << endl;
            }
            else if (token.type == TokenType::UNKNOWN) {
                cout << "Karakter tidak dikenal '" << token.value << "' pada
Baris " << token.line << ", Kolom " << token.column << endl;
            }
            else {
                cout << typeName << endl;
            }
            token = lexer.getNextToken();
        }
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << endl;
        return 1;
    }
    return 0;
}
```

Program akan dimulai dengan membaca *source code* Arion dari file yang berekstensi .txt yang diberikan melalui argumen *command line*. Objek Lexer akan diinisialisasi dengan *path* dari file tersebut, yang membaca seluruh isi file dan membangun tabel keyword. Tokenisasi akan berlangsung dengan memanggil getNextToken() secara berulang, di mana setiap pemanggilan akan melewati *whitespace*, dan melakukan *scanning* per karakter dengan menggunakan DFA, dan mengembalikan satu token.

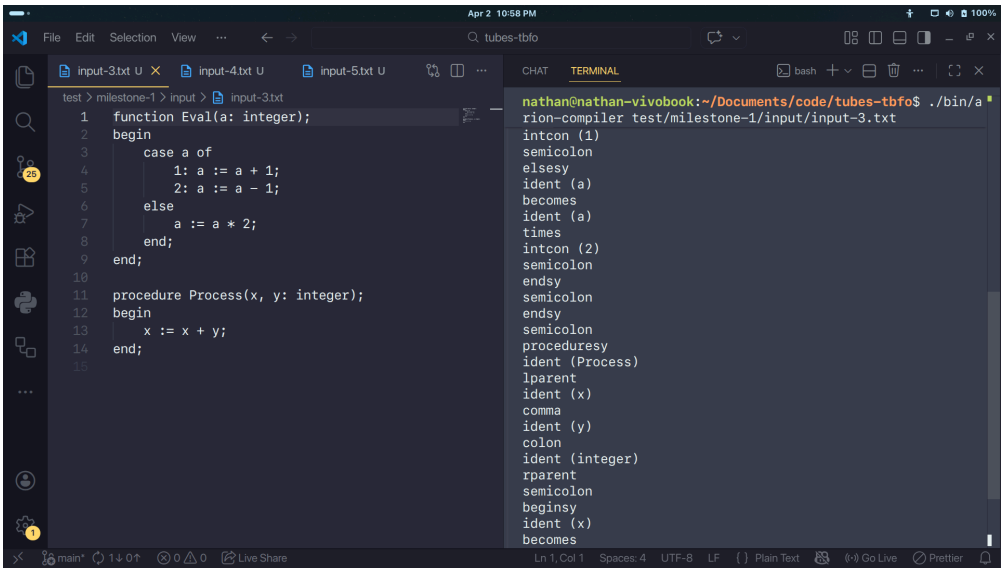
Hasil dari tokenisasi akan ditampilkan dalam format yang berbeda sesuai dengan jenis token: untuk IDENT, INTCON, dan REALCON, akan ditampilkan dengan tanda kutip typeNama (value), untuk UNKNOWN, akan ditampilkan pesan error beserta informasi dari posisi baris dan kolom, serta untuk token penanda lainnya seperti COMMENT, akan ditampilkan nama tokennya saja. Proses ini akan terus berlanjut sampai getNextToken() mengembalikan token END_OF_FILE.

BAB 3
Pengujian

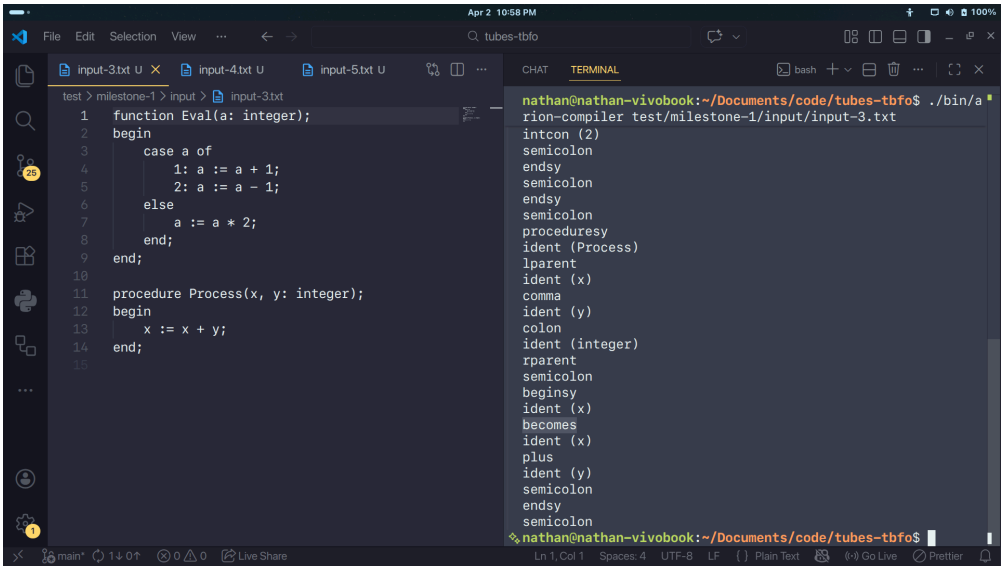
input-1.txt	
input-2.txt	

input-3.txt	

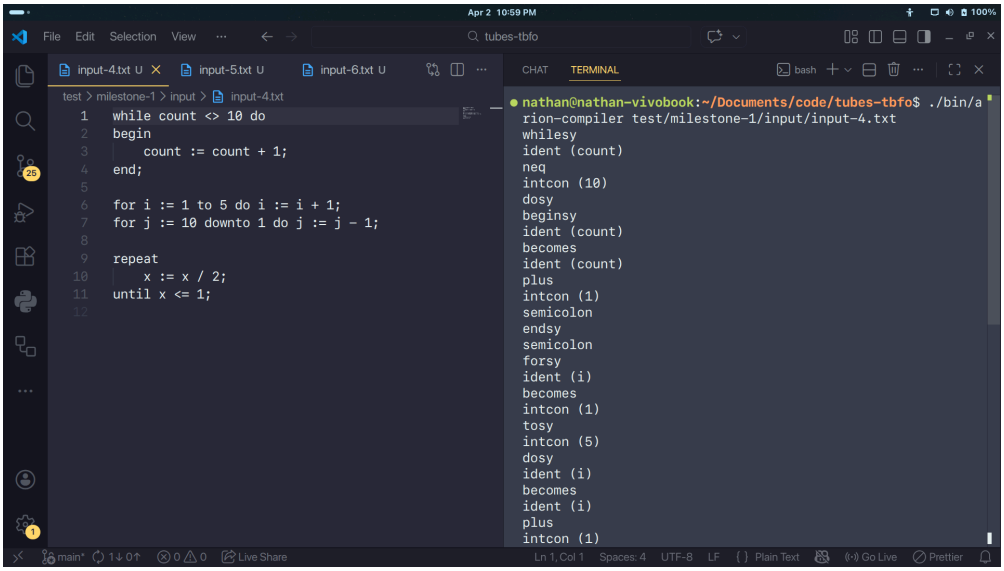
Apr 2 10:58 PM
File Edit Selection View
tubes-tbfo
input-3.txt U
input-4.txt U
input-5.txt U
test > milestone-1 > input > input-3.txt
function Eval(a: integer);
begin
case a of
1: a := a + 1;
2: a := a - 1;
else
a := a * 2;
end;
end;
procedure Process(x, y: integer);
begin
x := x + y;
end;
CHAT
TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-3.txt
functionsy
ident (Eval)
lparent
ident (a)
colon
ident (integer)
rparent
semicolon
beginsy
casesy
ident (a)
ofsy
intcon (1)
colon
ident (a)
becomes
ident (a)
plus
intcon (1)
semicolon
intcon (2)
colon
ident (a)
becomes
ident (a)
minus
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

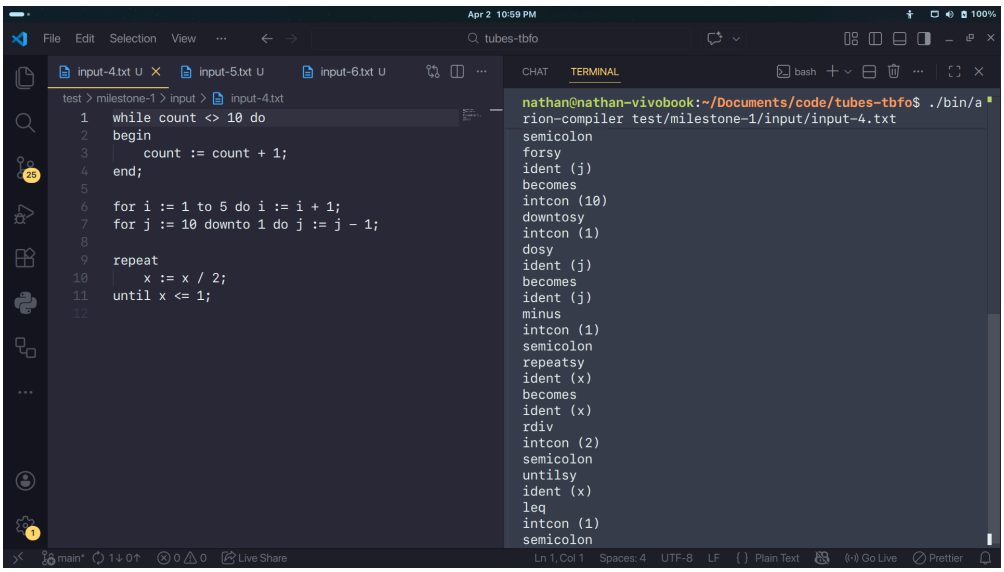


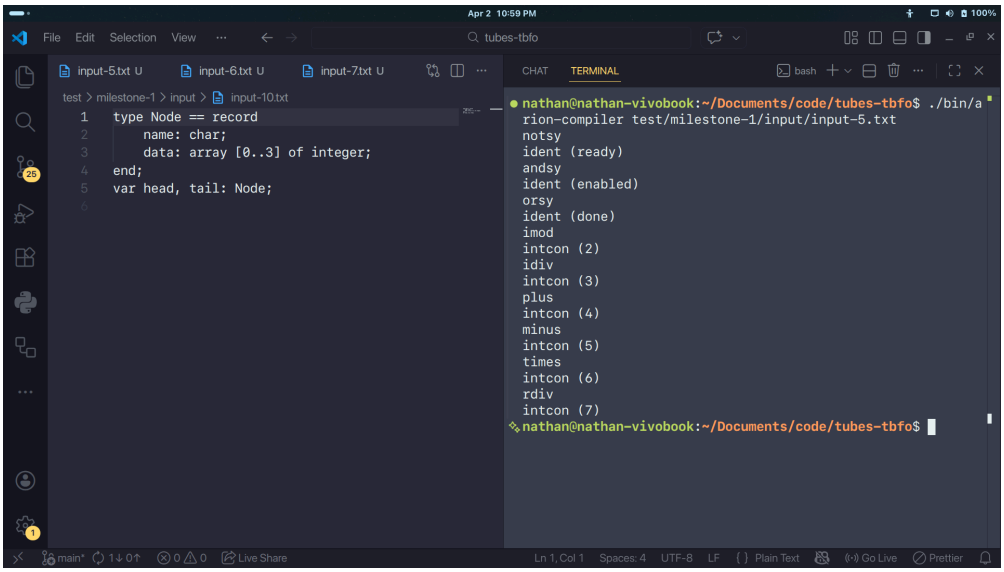
Apr 2 10:58 PM
File Edit Selection View
tubes-tbfo
input-3.txt U
input-4.txt U
input-5.txt U
test > milestone-1 > input > input-3.txt
function Eval(a: integer);
begin
case a of
1: a := a + 1;
2: a := a - 1;
else
a := a * 2;
end;
end;
procedure Process(x, y: integer);
begin
x := x + y;
end;
CHAT
TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-3.txt
intcon (1)
semicolon
elsesy
ident (a)
becomes
ident (a)
times
intcon (2)
semicolon
endsy
semicolon
endsy
semicolon
proceduresy
ident (Process)
lparent
ident (x)
comma
ident (y)
colon
ident (integer)
rparent
semicolon
beginsy
ident (x)
becomes
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

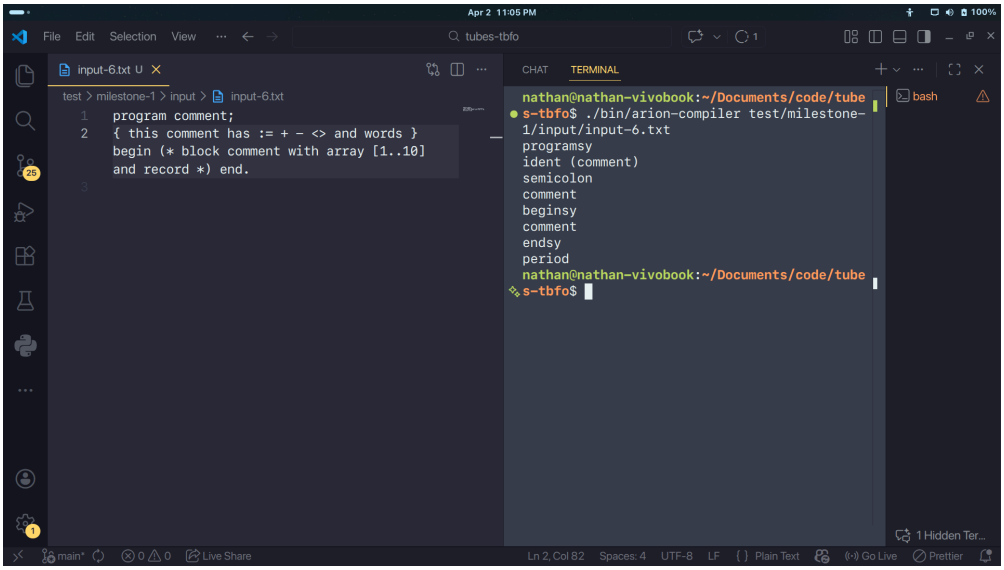
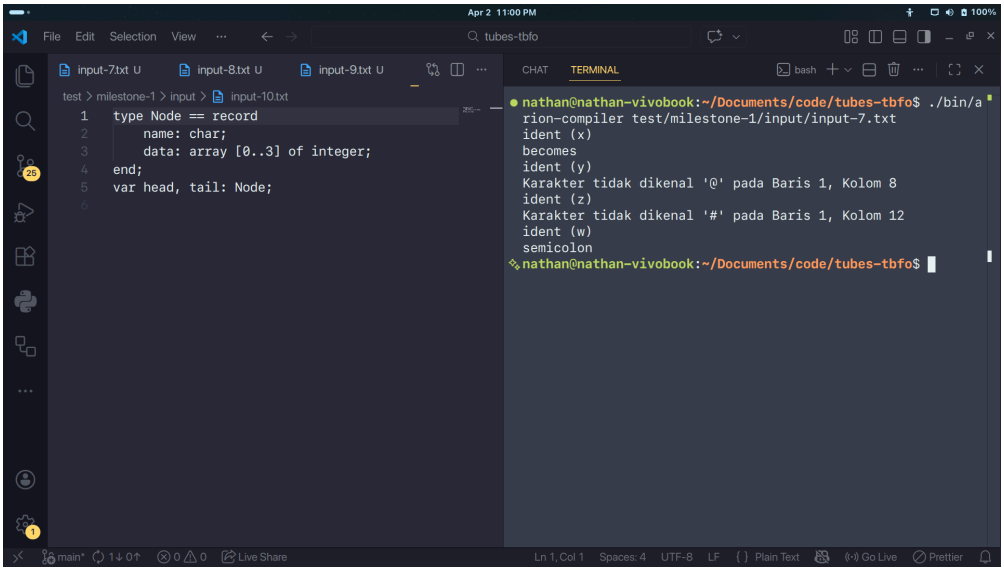

Apr 2 10:58 PM
File Edit Selection View
tubes-tbfo
input-3.txt U
input-4.txt U
input-5.txt U
test > milestone-1 > input > input-3.txt
function Eval(a: integer);
begin
case a of
1: a := a + 1;
2: a := a - 1;
else
a := a * 2;
end;
end;
procedure Process(x, y: integer);
begin
x := x + y;
end;
CHAT
TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-3.txt
intcon (2)
semicolon
endsy
semicolon
endsy
semicolon
proceduresy
ident (Process)
lparent
ident (x)
comma
ident (y)
colon
ident (integer)
rparent
semicolon
beginsy
ident (x)
becomes
ident (x)
plus
ident (y)
semicolon
endsy
semicolon
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

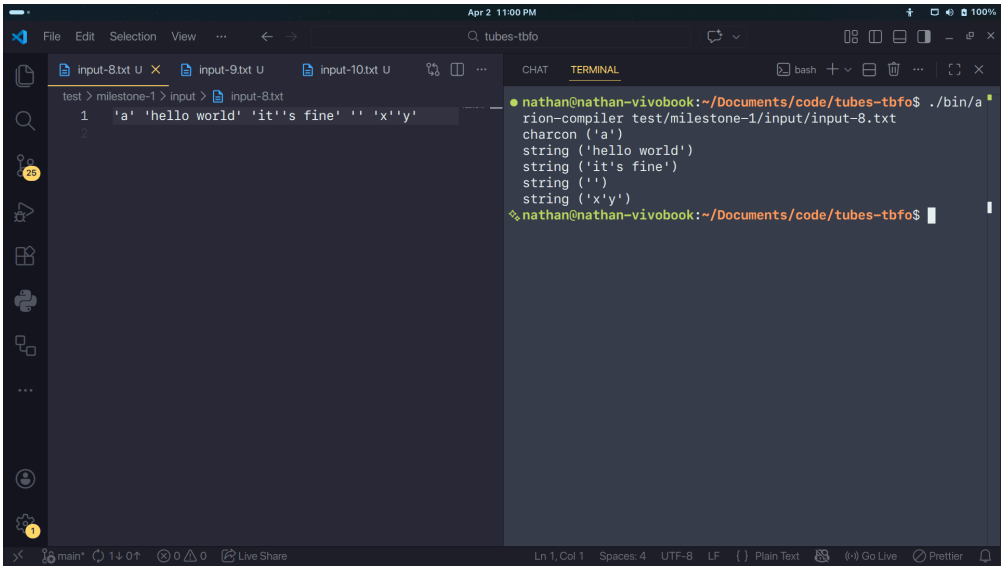
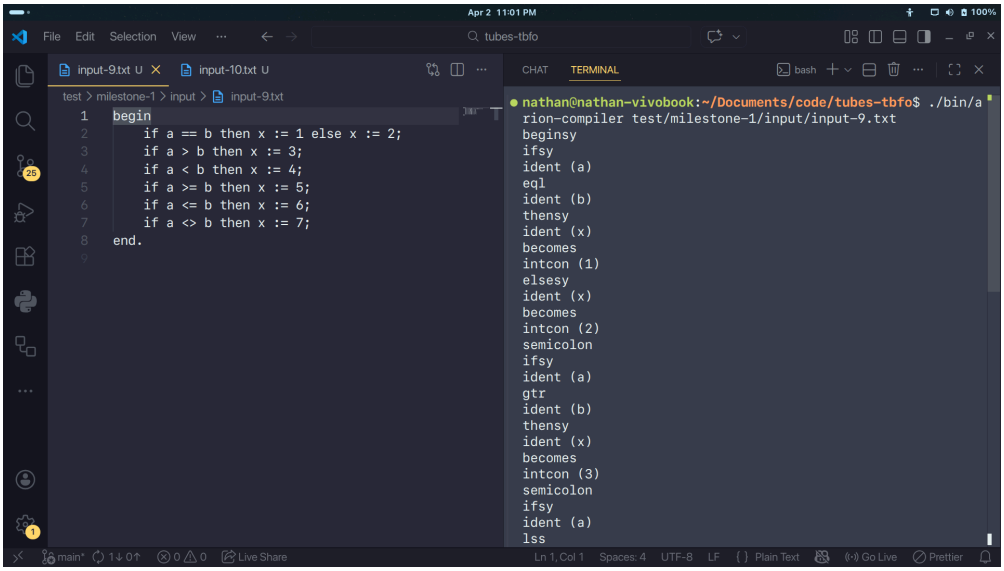
input-4.txt

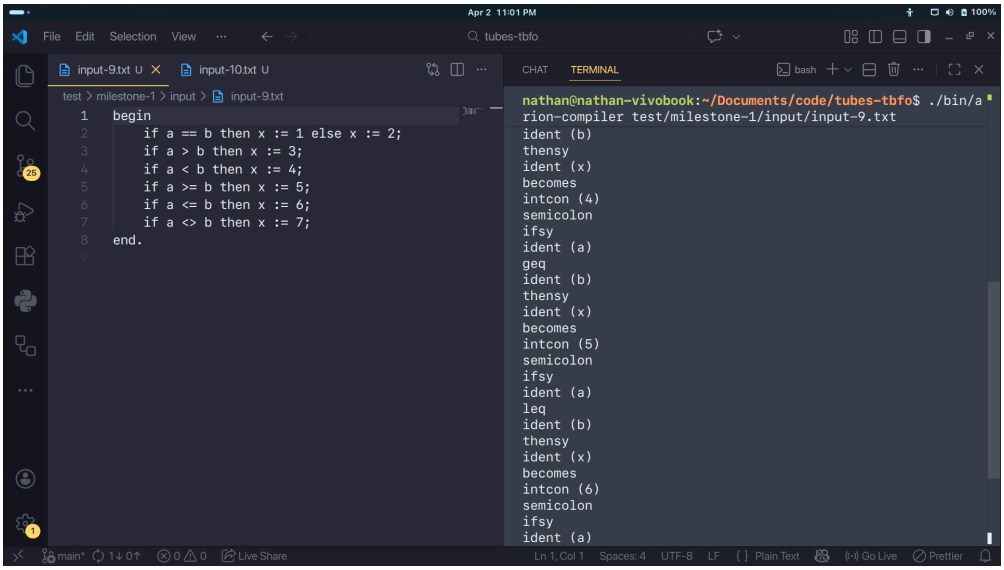

Apr 2 10:59 PM
File Edit Selection View
tubes-tbfo
input-4.txt U
input-5.txt U
input-6.txt U
test > milestone-1 > input > input-4.txt
while count <> 10 do
begin
count := count + 1;
end;
for i := 1 to 5 do i := i + 1;
for j := 10 downto 1 do j := j - 1;
repeat
x := x / 2;
until x <= 1;
CHAT
TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-4.txt
whilesy
ident (count)
neq
intcon (10)
dosy
beginsy
ident (count)
becomes
ident (count)
plus
intcon (1)
semicolon
endsy
semicolon
forsy
ident (i)
becomes
intcon (1)
tosy
intcon (5)
dosy
ident (i)
becomes
ident (i)
plus
intcon (1)
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

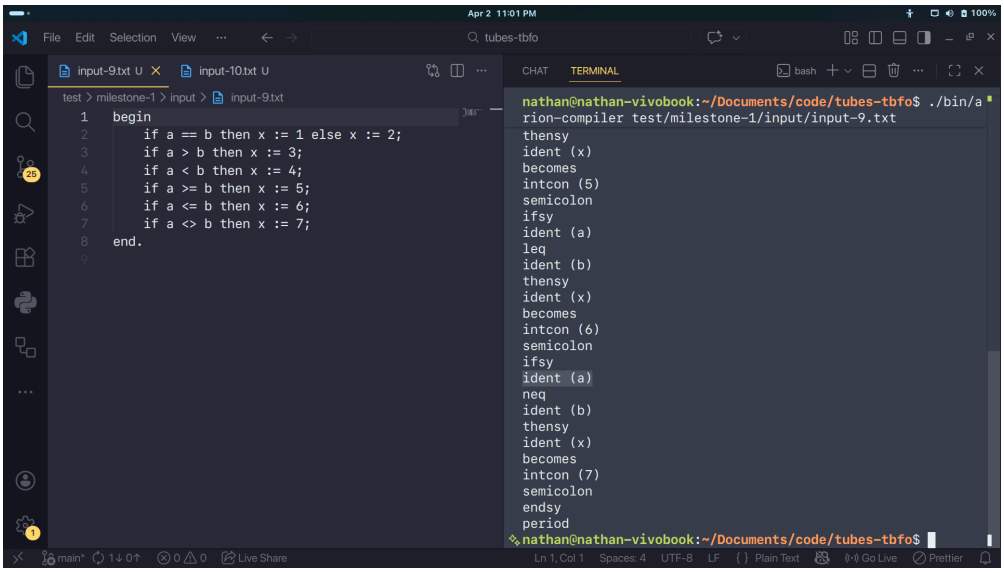
	 Apr 2 10:59 PM
	File Edit Selection View
	tubes-tbfo
	input-4.txt U
	input-5.txt U
	input-6.txt U
	test > milestone-1 > input > input-4.txt
	while count <> 10 do
	begin
	count := count + 1;
	end;
	for i := 1 to 5 do i := i + 1;
	for j := 10 downto 1 do j := j - 1;
	repeat
	x := x / 2;
	until x <= 1;
	CHAT TERMINAL
	nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
	rion-compiler test/milestone-1/input/input-4.txt
	semicolon
	forsy
	ident (j)
	becomes
	intcon (10)
	downtosy
	intcon (1)
	dosy
	ident (j)
	becomes
	ident (j)
	minus
	intcon (1)
	semicolon
	repeatsy
	ident (x)
	becomes
	ident (x)
	rdiv
	intcon (2)
	semicolon
	untilsy
	ident (x)
	leq
	intcon (1)
	semicolon

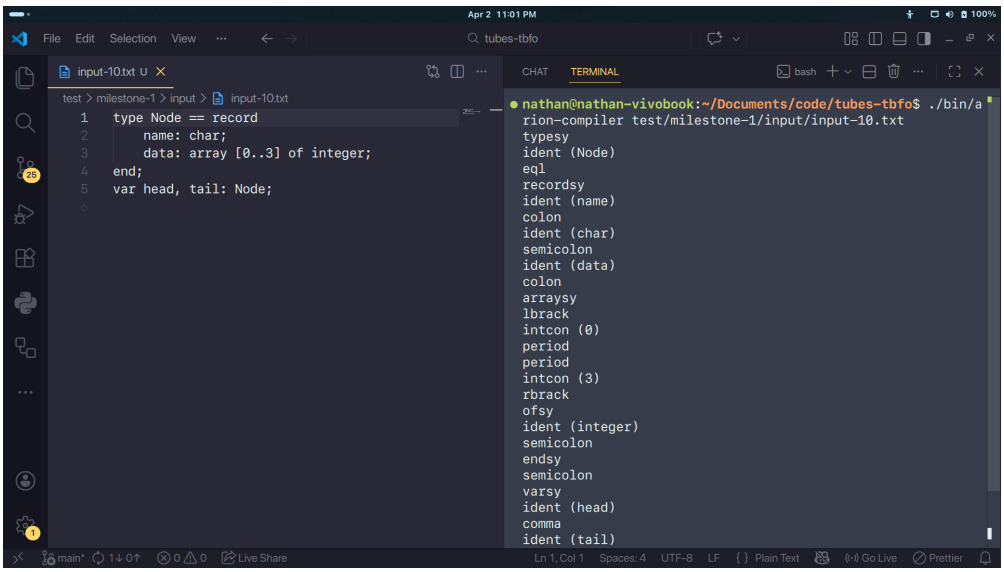
input-5.txt	 Apr 2 10:59 PM File Edit Selection View tubes-tbfo input-5.txt U input-6.txt U input-7.txt U test > milestone-1 > input > input-10.txt type Node == record name: char; data: array [0..3] of integer; end; var head, tail: Node; CHAT TERMINAL nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a rion-compiler test/milestone-1/input/input-5.txt notsy ident (ready) andsy ident (enabled) orsy ident (done) imod intcon (2) idiv intcon (3) plus intcon (4) minus intcon (5) times intcon (6) rdiv intcon (7) nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$

input-6.txt	
input-7.txt	

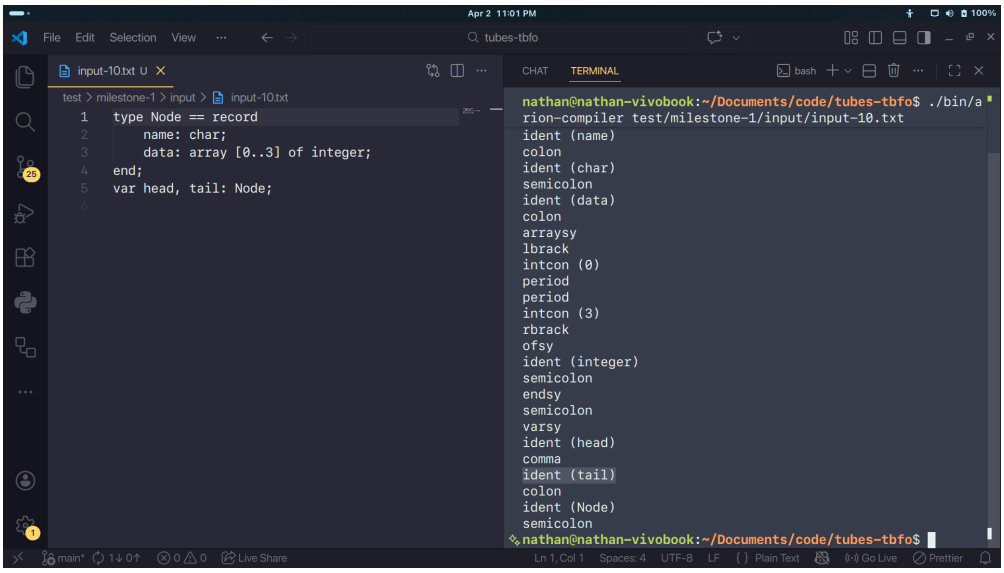
input-8.txt	
input-9.txt	


Apr 2 11:01 PM
File Edit Selection View
tubes-tbfo
input-9.txt U
input-10.txt U
test > milestone-1 > input > input-9.txt
begin
if a == b then x := 1 else x := 2;
if a > b then x := 3;
if a < b then x := 4;
if a >= b then x := 5;
if a <= b then x := 6;
if a <> b then x := 7;
end.
CHAT TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-9.txt
ident (b)
thensy
ident (x)
becomes
intcon (4)
semicolon
ifsy
ident (a)
geq
ident (b)
thensy
ident (x)
becomes
intcon (5)
semicolon
ifsy
ident (a)
leq
ident (b)
thensy
ident (x)
becomes
intcon (6)
semicolon
ifsy
ident (a)
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier


Apr 2 11:01 PM
File Edit Selection View
tubes-tbfo
input-9.txt U
input-10.txt U
test > milestone-1 > input > input-9.txt
begin
if a == b then x := 1 else x := 2;
if a > b then x := 3;
if a < b then x := 4;
if a >= b then x := 5;
if a <= b then x := 6;
if a <> b then x := 7;
end.
CHAT TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-9.txt
thensy
ident (x)
becomes
intcon (5)
semicolon
ifsy
ident (a)
leq
ident (b)
thensy
ident (x)
becomes
intcon (6)
semicolon
ifsy
ident (a)
neq
ident (b)
thensy
ident (x)
becomes
intcon (7)
semicolon
endsy
period
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

input-10.txt	

Apr 2 11:01 PM
File Edit Selection View
tubes-tbfo
input-10.txt U
test > milestone-1 > input > input-10.txt
type Node == record
name: char;
data: array [0..3] of integer;
end;
var head, tail: Node;
CHAT TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-10.txt
typesy
ident (Node)
eql
recordsy
ident (name)
colon
ident (char)
semicolon
ident (data)
colon
arraysy
lbrack
intcon (0)
period
period
intcon (3)
rbrack
ofsy
ident (integer)
semicolon
endsy
semicolon
varsy
ident (head)
comma
ident (tail)
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier



Apr 2 11:01 PM
File Edit Selection View
tubes-tbfo
input-10.txt U
test > milestone-1 > input > input-10.txt
type Node == record
name: char;
data: array [0..3] of integer;
end;
var head, tail: Node;
CHAT TERMINAL
bash
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$ ./bin/a
rion-compiler test/milestone-1/input/input-10.txt
ident (name)
colon
ident (char)
semicolon
ident (data)
colon
arraysy
lbrack
intcon (0)
period
period
intcon (3)
rbrack
ofsy
ident (integer)
semicolon
endsy
semicolon
varsy
ident (head)
comma
ident (tail)
colon
ident (Node)
semicolon
nathan@nathan-vivobook:~/Documents/code/tubes-tbfo$
main*
Live Share
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Plain Text
Go Live
Prettier

BAB 4
Kesimpulan dan Saran

4.1 Kesimpulan

Pada Milestone 1 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata ini, kami berhasil mengimplementasikan lexical analyzer (lexer) untuk bahasa pemrograman Orion menggunakan pendekatan Deterministic Finite Automata (DFA). Implementasi ini mencakup proses pengenalan berbagai jenis token, seperti keyword, identifier, operator, bilangan integer dan real, literal string dan character, komentar, serta tanda baca sesuai spesifikasi yang digunakan.

4.2 Saran

Kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

BillyO	rajin - rajin cek qna
Nathan	笑死我了。
Azri	Speknya tolong ditambah lagi penjelasannya kak
Rasyad	Pake go besok 🤤

Lampiran

Repository: https://github.com/AzriVz/CTF-Tubes-IF2224-2026

Diagram Transisi DFA:
https://drive.google.com/file/d/1Q8MlnukMm2jU2dEt5GNVJjzSQ2MCpJng/view?usp=sharing

Tabel Pembagian Tugas:

Nama	NIM	Workload	Persentase
Billy Ontoseno Irawan	13524121	Laporan (Bab 1)	20%
Nathaniel Christian	13524122	Laporan (Bab 3 dan 4) Test cases	20%
Azri Arzaq Pohan	13523139	Lexer.cpp Lexer.hpp main.cpp Laporan (Bab 2)	35%
Rasyad Satyatma	13524142	Diagram Transisi DFA	25%

Referensi

CS 241 Lecture 9 - Maximal Munch and Context Free Grammars With thanks to Brad Lushman, Troy Vasiga and Kevin Lanctot